

Does Social Interaction Modeling Actually Help?

A Controlled Comparison of Graph-Based and Interaction-Agnostic Pedestrian Trajectory Prediction

Sara Babić

*Machine Learning and Artificial Intelligence
Faculty of Technical Sciences, University of Novi Sad
Novi Sad, Serbia*

Abstract—Socially-aware robot navigation requires predicting where nearby pedestrians will be several seconds into the future. The dominant assumption in this literature is that explicitly modeling social interaction, typically through graph neural networks or attention, is the key ingredient for accurate prediction. We show that on the standard ETH-UCY benchmark this assumption is not supported once the comparison is properly controlled. We implement three models under an identical data pipeline and evaluation protocol: a vanilla LSTM baseline trained with an L_2 objective, the graph-based Social-STGCNN trained with a bivariate Gaussian negative log-likelihood, and, as a control, the same LSTM equipped with an identical probabilistic head and loss. The conventional comparison between the first two models is confounded, since they differ simultaneously in architecture and in training objective. When only the architecture differs, the interaction-agnostic model attains an average displacement error of 0.405 m against 0.415 m for the graph model, i.e. no measurable advantage from social modeling; the graph model wins on only one of five scenes. Both models reproduce published performance, confirming that this is a property of the benchmark rather than of the implementation. What does yield a substantial improvement is uncertainty modeling (0.552 $\rightarrow$ 0.405 m, a 27% reduction), obtained by predicting a distribution instead of a point, and parameter efficiency: the graph model matches the LSTM using 7,563 versus 199,106 parameters, a 26 $\times$ reduction. We additionally describe a block-diagonal graph batching scheme that accelerates graph training by 24 $\times$. Our results suggest that for socially-aware navigation the practically valuable output is a calibrated distribution over future positions rather than a structured interaction model.

Index Terms—trajectory prediction, socially-aware navigation, graph neural networks, LSTM, uncertainty estimation, ETH-UCY, controlled comparison

I. INTRODUCTION

A robot sharing space with people cannot treat them as static obstacles. Classical planners such as A*, RRT or DWA optimize path length or traversal time against a snapshot of the environment, and consequently produce manoeuvres that are collision-free yet socially unacceptable: cutting through a conversing group, passing uncomfortably close, or freezing in place when every direction appears blocked, the well-known *freezing robot problem* [1]. Making a

planner socially aware therefore requires a perception layer that answers a predictive question: where will each person be several seconds from now, and with what confidence [2]?

This has motivated a large body of work on pedestrian trajectory prediction, and a clear methodological trend within it. Since Social-LSTM [3] introduced social pooling, successive architectures have modeled inter-agent interaction ever more explicitly - through generative modeling [4], graph convolution [5], or heterogeneous graph structures [6]. The implicit premise is that interaction modeling is where predictive accuracy comes from.

This paper questions that premise, not by proposing yet another architecture, but by constructing a comparison that isolates it. Our starting observation is that the usual comparison in this literature is confounded. An interaction-agnostic baseline is typically trained with a deterministic L_2 objective, whereas the proposed social model is probabilistic and trained with a negative log-likelihood, then evaluated under a best-of- K sampling protocol. The two models therefore differ in *two* variables simultaneously - architecture and training objective - and any observed difference cannot be attributed to social modeling alone.

We remove the confound by introducing a third model: the same interaction-agnostic LSTM, given an identical bivariate Gaussian head, trained with the identical loss, selected by the identical criterion, and evaluated under the identical protocol as the graph model. Only the architecture then differs.

Our contributions are:

- A controlled experiment showing that on ETH-UCY, at the standard 4.8 s horizon, graph-based social modeling yields no average accuracy advantage over an interaction-agnostic architecture (0.415 m vs. 0.405 m ADE).
- An attribution of where the improvement actually comes from: uncertainty modeling reduces ADE by 27%, and the graph architecture delivers a 26 $\times$ reduction in parameter count at equal accuracy.
- A block-diagonal graph batching scheme that makes spatio-temporal graph training 24 $\times$ faster without altering graph propagation.

- A complete, reproducible implementation in which all three models share one data pipeline and one evaluation module, so that no measured difference can originate from the data or the metric code.

II. RELATED WORK

Hand-crafted interaction models. The Social Force model [7] represented pedestrians as particles subject to attractive and repulsive potentials. Such models are interpretable but require manual parameter tuning and generalize poorly across environments.

Recurrent models. Social-LSTM [3] paired a per-pedestrian LSTM with a social pooling layer aggregating hidden states of spatial neighbours. Social-GAN [4] introduced adversarial training to produce multiple socially plausible futures, and established the best-of- K evaluation protocol that subsequent probabilistic work adopted.

Graph-based models. Social-STGCNN [5] replaced pooling with an explicit graph in which nodes are pedestrians and edge weights encode spatial proximity, combining graph convolution [8] with temporal convolution in the spirit of spatio-temporal graph networks for traffic forecasting [9]. Trajectron++ [6] extended this to heterogeneous agents and dynamic constraints.

Critical evaluations. Schöller *et al.* [10] showed that a constant velocity model outperforms many published social architectures on ETH-UCY, arguing that benchmark performance is dominated by individual motion inertia. Our work is complementary: rather than comparing learned models against a non-learned baseline, we compare two learned models that are matched in every respect except the presence of interaction modeling.

III. PROBLEM FORMULATION AND DATASET

A. Problem statement

For pedestrian i we observe a history of positions $X_i = \{(x_i^t, y_i^t)\}_{t=1}^{T_{obs}}$ and predict the future $\hat{Y}_i = \{(\hat{x}_i^t, \hat{y}_i^t)\}_{t=T_{obs}+1}^{T_{obs}+T_{pred}}$. Following standard practice we set $T_{obs} = 8$ and $T_{pred} = 12$. At the 2.5 Hz recording rate of the benchmark this corresponds to observing 3.2 s and predicting 4.8 s.

B. Dataset

We use ETH-UCY [11], [12], comprising five scenes recorded from a bird’s-eye view with positions expressed in world coordinates in metres. Metric coordinates make the error measures directly interpretable as physical distances, which is precisely what a planner requires.

Evaluation follows the *leave-one-scene-out* protocol: for each scene, models are trained on the remaining four and tested on the held-out scene. This measures generalization to an entirely new physical space rather than to new pedestrians in a familiar one, which is the realistic condition for a robot entering an unmapped environment. We adopt the standardized split distributed with [5], identical to

TABLE I
DATASET STATISTICS UNDER THE LEAVE-ONE-SCENE-OUT PROTOCOL. A SEQUENCE IS ONE 20-FRAME WINDOW; A TRAJECTORY IS ONE PEDESTRIAN PRESENT IN ALL 20 FRAMES OF THAT WINDOW.

Scene	Train		Validation		Test	
	Seq.	Traj.	Seq.	Traj.	Seq.	Traj.
ETH	3,283	30,307	733	5,422	253	364
HOTEL	3,118	29,676	688	5,203	445	1,197
UNIV	2,719	9,874	622	2,800	947	24,334
ZARA1	2,889	28,577	671	5,184	705	2,356
ZARA2	2,681	26,076	590	4,262	998	5,910

that of [4], so that our numbers are directly comparable to published results. Table I reports the resulting statistics.

The asymmetry in Table I is a direct consequence of the protocol: UNIV is a dense scene, so holding it out both removes the densest training examples and produces a test set averaging 25.7 pedestrians per window, against 1.4 for ETH.

IV. METHODOLOGY

A. Preprocessing

A sliding window of $T_{obs} + T_{pred} = 20$ consecutive frames traverses each recording. Within a window we retain only pedestrians present in all 20 frames, matching the reference protocol and avoiding padding semantics that would differ from published work.

Crucially, the models operate on *relative displacements* $\Delta p^t = p^t - p^{t-1}$ rather than absolute coordinates. Each scene has its own coordinate frame, so a model consuming absolute positions learns where people stand in the training scenes, which is worthless information in an unseen scene. Displacements make the model translation invariant, so that it learns how people move. Predicted displacements are converted back to positions by cumulative summation from the last observed point.

Because scenes contain variable numbers of pedestrians, all pedestrians in a batch are concatenated along a single dimension while a companion tensor records per-scene boundaries. The LSTM ignores these boundaries entirely; the graph model uses them to determine which pedestrians belong to the same graph. A single loader therefore serves all three models, guaranteeing that they observe byte-identical data.

B. Baseline: vanilla LSTM

The baseline is an encoder–decoder LSTM [13]. A linear layer embeds each displacement into a 64-dimensional space; an encoder LSTM with 128 hidden units compresses the observed motion into a hidden state; a decoder LSTM autoregressively emits 12 future displacements, feeding each output back as the next input. No pedestrian ever observes another, which is precisely the point, as this isolates how much of the task is explained by an agent’s own inertia. The model has 199,106 parameters and is trained with the mean

Euclidean error over displacements, a quantity directly aligned with the reported metric. Gradients are clipped to unit norm, as the autoregressive decoder propagates through twelve chained steps.

C. Graph model: Social-STGCNN

Social-STGCNN [5] represents each timestep as a graph whose nodes are pedestrians. Edge weights are inverse distances, $a_{ij} = \|p_i - p_j\|_2^{-1}$, so that nearby pedestrians influence one another more strongly, a direct formalization of personal space. The adjacency matrix is symmetrically normalized following [8],

$$\hat{A} = D^{-1/2} (A + I) D^{-1/2}, \quad (1)$$

where the added identity provides self-loops so that a node retains its own features during aggregation.

We note one deliberate deviation from the reference implementation of [5], which normalizes the adjacency matrix using the normalized graph Laplacian $L = I - D^{-1/2} A D^{-1/2}$ rather than the normalized adjacency $\hat{A}$ used here. Since $L = I - \hat{A}$, the two operators differ, and our reported figures correspond to the formulation above. We adopt $\hat{A}$ because it is the standard GCN propagation rule [8]; our results reproduce the published performance of the original model, so the choice is not detrimental.

Each ST-GCNN block applies a spatial graph convolution followed by a temporal convolution along the time axis. A Time-Extrapolator CNN then maps the 8 observed steps to 12 predicted steps by convolving over the temporal dimension, producing all future steps in a single pass rather than recurrently.

The output at each step consists of the five parameters of a bivariate Gaussian, $(\mu_x, \mu_y, \sigma_x, \sigma_y, \rho)$, trained by minimizing the negative log-likelihood of the observed displacement. Numerical stability is enforced by parameterizing $\sigma = \exp(\cdot)$ and $\rho = \tanh(\cdot)$ with clamping, without which NLL training diverges in practice. The model has only 7,563 parameters.

D. Controlled comparison: probabilistic LSTM

The two models above differ in architecture *and* in training objective. A model trained with NLL has a mean that is not optimized for L_2 error, so evaluating that mean against an L_2 -trained baseline penalizes the graph model for reasons unrelated to social modeling. Conversely, comparing best-of-20 sampling against a single deterministic prediction compares twenty attempts with one. Neither comparison answers the question of interest.

We therefore introduce a control: the same vanilla LSTM, with its output head widened from two to five values, trained with the *identical* bivariate NLL loss, selected on the *identical* validation criterion, and evaluated under the *identical* best-of-20 protocol as the graph model. The loss function and sampling routine are literally the same code paths, so the protocols cannot silently diverge. The two models now differ in exactly one respect: whether the architecture can observe other pedestrians.

E. Efficient batched graph training

A direct implementation processes one scene per optimizer step, as in the reference implementation. With 3,283 sequences per epoch this issues thousands of very small GPU launches. Profiling the ETH training loop attributes 0.2 s per epoch to data loading, 9.0 s to adjacency construction and 34.1 s to the forward and backward passes, for approximately 62 s per epoch including validation, or about 13 hours for all five scenes.

We instead pack B scenes into a single graph whose adjacency matrix is block diagonal. Pedestrians from different scenes have no edge between them, so graph propagation is identical to processing the scenes separately, while the computation is issued as one GPU call. Combined with vectorized best-of- K sampling and periodic rather than per-epoch validation, this reduces training to 2.6 s per epoch, a 24 $\times$ speedup; all five scenes train in roughly 15 minutes.

We note for completeness that batch normalization now computes statistics over all nodes in the batch rather than over a single scene. This is standard mini-batch behaviour and generally stabilizes training, but it is not a bit-identical computation.

F. Hyperparameter optimization

Architecture and learning rate are selected using Optuna [14] with a Tree-structured Parzen Estimator sampler [15] and median pruning. The search space covers learning rate, the number of graph convolution blocks, the depth of the temporal decoder, the temporal kernel size and dropout. The objective is *validation* ADE; the test set is never consulted during the search, as selecting hyperparameters on test performance constitutes information leakage that silently inflates reported results.

V. EXPERIMENTAL SETUP

A. Metrics

We report the two standard measures. Average Displacement Error is the mean *Euclidean* distance between predicted and true positions over all predicted steps and all pedestrians,

$$\text{ADE} = \frac{1}{N T_{\text{pred}}} \sum_{i=1}^N \sum_{t=T_{\text{obs}}+1}^{T_{\text{obs}}+T_{\text{pred}}} \|\hat{p}_i^t - p_i^t\|_2, \quad (2)$$

and Final Displacement Error is the distance at the final predicted step,

$$\text{FDE} = \frac{1}{N} \sum_{i=1}^N \left\| \hat{p}_i^{T_{\text{obs}}+T_{\text{pred}}} - p_i^{T_{\text{obs}}+T_{\text{pred}}} \right\|_2. \quad (3)$$

Both are expressed in metres. Errors are accumulated as sums over pedestrians and divided once at the end, since averaging per-batch means would misweight batches containing different numbers of pedestrians.

B. Protocols

For probabilistic models we report both a *deterministic* figure, obtained from the mean of the predicted distribution, and the *best-of-20* figure standard in the literature, in which 20 trajectories are sampled and the per-pedestrian minimum is retained. Reporting only one of the two would mislead in opposite directions.

C. Implementation

All models are implemented in PyTorch and trained with Adam on a single NVIDIA RTX 4050 laptop GPU. The deterministic LSTM baseline is trained for 100 epochs; the probabilistic LSTM and Social-STGCNN are both trained for 250 epochs, so that the two models entering the controlled comparison receive an identical training budget. Checkpoints are selected by validation ADE in every case. The complete pipeline (three models, hyperparameter search and figures) runs in approximately 75 minutes.

VI. RESULTS

A. Validation against published results

Table II compares our models against published figures. Both fall within the published range; Social-STGCNN in fact surpasses its published average and its per-scene results on HOTEL, UNIV and ZARA2. This confirms that the protocol and evaluation are implemented correctly, which is the precondition for trusting the comparisons that follow.

B. The confounded comparison

Table III presents the comparison as it is conventionally made, between the L_2 -trained baseline and the deterministic output of the graph model. The graph model is marginally *worse* on average (0.563 vs. 0.552 m). As argued in Section IV-D, this comparison is confounded by the difference in training objective and should not be read as evidence about social modeling.

C. The controlled comparison

Table IV contains the paper’s central result. Both models now share an identical probabilistic head, loss function, selection criterion and evaluation protocol; the only difference is whether the architecture observes other pedestrians.

The interaction-agnostic model attains a better average ADE than the graph model (0.405 vs. 0.415 m). Social-STGCNN wins on only one scene, HOTEL; it loses on the remaining four. Fig. 1 shows that the two models also converge to indistinguishable validation error during training.

D. Where the improvement actually comes from

Two factors do produce substantial, measurable gains.

First, **uncertainty modeling**. Moving from a point prediction to a distribution reduces average ADE from 0.552 m to 0.405 m, a 27% improvement. This gain is obtained by the interaction-agnostic architecture and

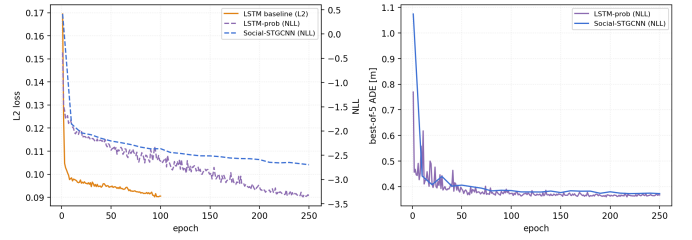


Fig. 1. Training dynamics on ZARA1. Left: training losses; L_2 and NLL objectives are plotted on separate axes as they are not on a common scale. Right: validation ADE for the two probabilistic models under the identical best-of-5 protocol, the only mutually comparable curves. They converge to indistinguishable error.

is therefore unrelated to the graph. The mechanism is multimodality: a model trained to emit a single point learns the conditional expectation, which for a pedestrian approaching a junction lies between the plausible outcomes, a location the pedestrian will never occupy.

Second, **parameter efficiency**. Social-STGCNN matches the LSTM using 7,563 against 199,106 parameters, a $26\times$ reduction (Table II). For an embedded robot controller with constrained memory and power budgets this is a substantive practical advantage even at equal accuracy, and it is the strongest claim for the graph architecture that our evidence supports.

E. Hyperparameter optimization

The TPE search over the graph model converged towards a wider temporal decoder (n_{txpcnn} from 5 to 7), a larger temporal kernel (3 to 5) and moderate dropout, improving validation ADE from 0.419 to 0.389 m on ZARA1 over 15 trials, 6 of which were pruned early. Notably the search did *not* favour additional graph convolution blocks, preferring deeper temporal processing, an independent signal consistent with our main finding.

Retraining with the selected configuration yielded a test ADE of 0.283 m, identical to the default configuration (0.283 m), with FDE improving only from 0.498 to 0.497 m. The validation gain therefore did not transfer to the test set. We report this rather than omit it: it indicates the default architecture was already near the ceiling for this dataset, and it illustrates why hyperparameters must be selected on validation and then verified on held-out data.

F. Qualitative analysis

Fig. 2 compares predictions on a ZARA1 scene containing seven pedestrians. Both models capture the dominant bidirectional flow; neither anticipates the mid-horizon direction changes visible in the ground truth, and their predictions are largely indistinguishable from one another, a visual counterpart to Table IV.

Fig. 3 shows the predicted motion density, obtained by drawing 2,000 trajectories from the learned distribution. The density widens with the prediction horizon, reflecting growing uncertainty, and separates into distinct lobes

TABLE II
COMPARISON WITH PUBLISHED RESULTS (AVERAGE ADE / FDE OVER THE FIVE SCENES, IN METRES) AND PARAMETER COUNTS.

Model	Ours	Published	Params
LSTM baseline (det.)	0.552 / 1.159	0.70 / 1.63 [4]	199,106
LSTM-prob (best-of-20)	0.405 / 0.761	n/a	199,493
Social-STGCNN (best-of-20)	0.415 / 0.784	0.44 / 0.75 [5]	7,563

TABLE III
CONFOUNDED COMPARISON: DETERMINISTIC PREDICTIONS FROM MODELS TRAINED WITH DIFFERENT OBJECTIVES (L_2 VS. NLL). ADE / FDE IN METRES; LOWER IS BETTER.

Scene	LSTM (L_2)	Social-STGCNN (NLL mean)
ETH	1.007 / 1.996	1.043 / 2.113
HOTEL	0.457 / 0.936	0.436 / 0.880
UNIV	0.577 / 1.278	0.568 / 1.202
ZARA1	0.402 / 0.886	0.419 / 0.904
ZARA2	0.316 / 0.699	0.350 / 0.761
Average	0.552 / 1.159	0.563 / 1.172

TABLE IV
Controlled comparison. BOTH MODELS USE AN IDENTICAL BIVARIATE GAUSSIAN HEAD, NLL LOSS AND BEST-OF-20 PROTOCOL; ONLY THE ARCHITECTURE DIFFERS: LSTM-PROB NEVER OBSERVES OTHER AGENTS, SOCIAL-STGCNN DOES. ADE / FDE IN METRES; LOWER IS BETTER.

Scene	LSTM-prob	Social-STGCNN
ETH	0.775 / 1.405	0.819 / 1.582
HOTEL	0.342 / 0.659	0.311 / 0.561
UNIV	0.411 / 0.816	0.412 / 0.803
ZARA1	0.276 / 0.497	0.283 / 0.498
ZARA2	0.221 / 0.428	0.251 / 0.474
Average	0.405 / 0.761	0.415 / 0.784

corresponding to the two opposing pedestrian flows. This field is directly usable as a cost map by a socially-aware planner, and it is the practical product of uncertainty modeling rather than of the graph structure.

VII. DISCUSSION

Our results indicate that on ETH-UCY, at a 4.8 s horizon, explicit graph-based interaction modeling does not improve average predictive accuracy relative to an architecture that cannot observe other agents. Three considerations argue that this reflects the benchmark rather than a defective implementation: both models reproduce published performance; all models share a single data pipeline, so no difference can originate from the data; and the loss, selection criterion and evaluation protocol are identical between the two compared models.

The most plausible explanation is that pedestrian motion over this horizon is dominated by inertia. People rarely change direction abruptly, both physically and because their intention persists over several seconds. Most of the

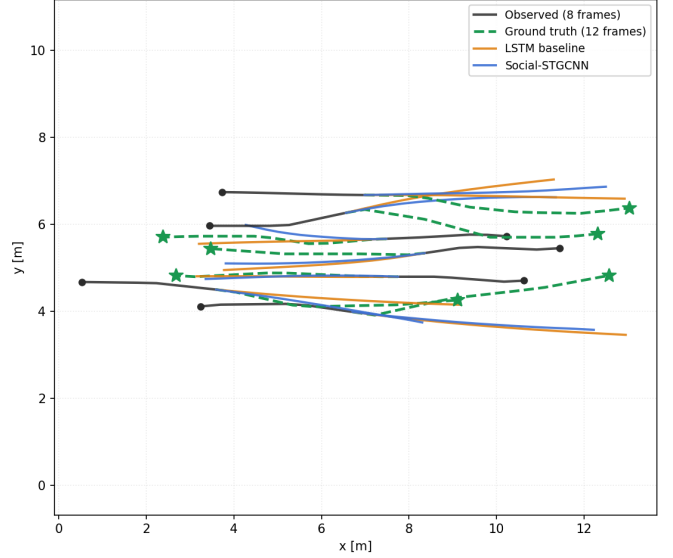


Fig. 2. Qualitative comparison on ZARA1. Observed history (black), ground truth future (green dashed), LSTM baseline (orange) and Social-STGCNN (blue). Stars mark the true final positions.

achievable error reduction therefore comes from extrapolating current heading and speed accurately, not from reasoning about interaction. This is consistent with [10], who reach a related conclusion using a constant velocity model. Situations in which interaction is decisive (last-moment collision avoidance, circumventing a group) are a small fraction of windows, so a model that handles them perfectly gains little in the average.

The single scene on which the graph model wins clearly, HOTEL, is consistent with this account. HOTEL contains many stationary or slowly-moving people near an entrance, precisely the regime in which "continue straight" is weakest and contextual information about neighbours is most informative. The graph helps where inertia does not.

For socially-aware navigation the implication is a refinement rather than a rejection of the original premise. What a planner most needs is a calibrated distribution over future positions, such as that in Fig. 3, and this is supplied by either architecture. Where the graph model retains a decisive advantage is deployment cost: equivalent accuracy at $26\times$ fewer parameters.

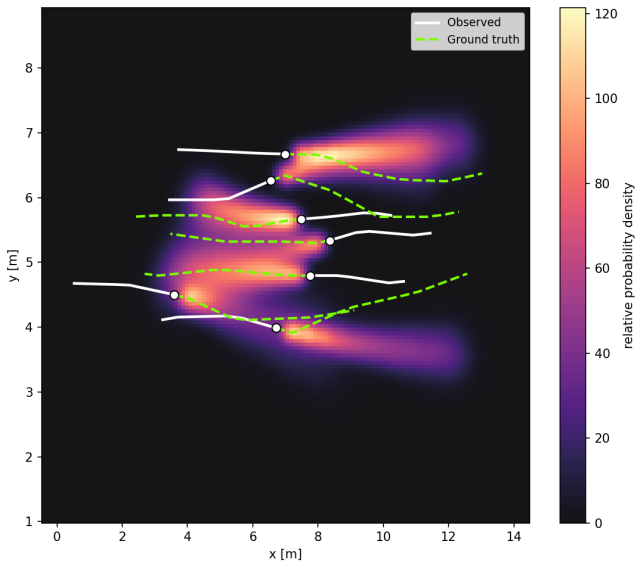


Fig. 3. Predicted motion density on ZARA1 from 2,000 samples of the learned distribution. White: observed history; green dashed: ground truth. Uncertainty grows with the horizon and separates by direction of travel.

VIII. LIMITATIONS AND FUTURE WORK

The models consume no map of the environment and will therefore predict trajectories through walls when the current heading points that way. Groups are not modeled explicitly. Perfect detection and tracking are assumed, whereas a deployed robot receives noisy detections with identity switches. The best-of-20 protocol is optimistic: a robot must act on a single belief rather than select the most fortunate of twenty samples. Finally, the hyperparameter search was conducted on a single scene, and the step from prediction to socially-aware planning is not implemented.

The most direct test of our interpretation would extend the prediction horizon beyond 8–10 s, where inertia should cease to dominate and interaction modeling would be expected to show a clearer advantage. Adding a semantic map would eliminate physically impossible predictions, and integrating the predicted density into a local planner would allow the practical value of the distribution to be measured directly.

IX. CONCLUSION

We examined whether explicit social interaction modeling improves pedestrian trajectory prediction, using a comparison controlled for training objective and evaluation protocol. On ETH-UCY at a 4.8 s horizon it does not: a graph-based model attains 0.415 m ADE against 0.405 m for an architecture that never observes other pedestrians. The improvements that are real lie elsewhere: uncertainty modeling reduces error by 27%, and the graph architecture achieves equivalent accuracy with $26\times$ fewer parameters. We conclude that for socially-aware navigation the valuable output is a calibrated distribution over future positions,

and that claims about the benefit of interaction modeling should be substantiated with comparisons in which the training objective is held fixed.

REPRODUCIBILITY

The complete implementation, trained checkpoints, hyperparameter search logs and the scripts that generate every table and figure in this paper are available in the project repository.

REFERENCES

- [1] P. Trautman and A. Krause, “Unfreezing the robot: Navigation in dense, interacting crowds,” *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, pp. 797–803, 2010.
- [2] C. Mavrogiannis, F. Baldini, A. Wang, D. Zhao, P. Trautman, A. Steinfeld, and J. Oh, “Core challenges of social robot navigation: A survey,” *ACM Transactions on Human-Robot Interaction*, vol. 12, no. 3, pp. 1–39, 2023.
- [3] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, “Social LSTM: Human trajectory prediction in crowded spaces,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.
- [4] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, “Social GAN: Socially acceptable trajectories with generative adversarial networks,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2255–2264.
- [5] A. Mohamed, K. Qian, M. Elhoseiny, and C. Claudel, “Social-STGCNN: A social spatio-temporal graph convolutional neural network for human trajectory prediction,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 14 424–14 432.
- [6] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, “Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data,” in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 683–700.
- [7] D. Helbing and P. Molnar, “Social force model for pedestrian dynamics,” *Physical Review E*, vol. 51, no. 5, pp. 4282–4286, 1995.
- [8] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.
- [9] B. Yu, H. Yin, and Z. Zhu, “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting,” in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2018, pp. 3634–3640.
- [10] C. Schöller, V. Aravantinos, F. Lay, and A. Knoll, “What the constant velocity model can teach us about pedestrian motion prediction,” *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1696–1703, 2020.
- [11] S. Pellegrini, A. Ess, K. Schindler, and L. Van Gool, “You’ll never walk alone: Modeling social behavior for multi-target tracking,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2009, pp. 261–268.
- [12] A. Lerner, Y. Chrysanthou, and D. Lischinski, “Crowds by example,” *Computer Graphics Forum*, vol. 26, no. 3, pp. 655–664, 2007.
- [13] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [14] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.
- [15] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2011, pp. 2546–2554.